

1.1 INTRODUCTION

Computers are classified according to **functionality, physical size and purpose**. Functionality, Computers could be **analog, digital or hybrid**. Digital computers process data that is in discrete form whereas analog computers process data that is continuous in nature. Hybrid computers on the other hand can process data that is both discrete and continuous. In digital computers, the user input is first converted and transmitted as electrical pulses that can be represented by two unique states ON and OFF. The ON state may be represented by a "1" and the off state by a "0". The sequence of ON'S and OFF'S forms the electrical signals that the computer can understand. A digital signal rises suddenly to a peak voltage of +1 for some time then suddenly drops -1 level on the other hand an analog signal rises to +1 and then drops to -1 in a continuous version.

Although the two graphs look different in their appearance, notice that they repeat themselves at equal time intervals. Electrical signals or waveforms of this nature are said to be periodic. Generally, a periodic wave representing a signal can be described using the following parameters

- Amplitude(A)
- Frequency(f)
- periodic time(T)
 - ▲ **Amplitude (A):** this is the maximum displacement that the waveform of an electric signal can attain.
 - ▲ **Frequency (f):** is the number of cycles made by a signal in one second. It is measured in hertz. 1 hertz is equivalent to 1 cycle/second.
 - ▲ **Periodic time (T):** the time taken by a signal to complete one cycle is called periodic time. Periodic time is given by the formula $T=1/f$, where f is the frequency of the wave.
 - ▲ When a digital signal is to be sent over analog telephone lines e.g. e-mail, it has to be converted to analog signal. This is done by connecting a device called a **modem** to the digital computer. This process of converting a digital signal to an analog signal is known as **modulation**. On the receiving end, the incoming analog signal is converted back to digital form in a process known as **demodulation**.

Concepts of Data Representation in Digital Computers

Data and instructions cannot be entered and processed directly into computers using human language. Any type of data be it numbers, letters, special symbols, sound or pictures must first be converted into machine-readable form i.e. binary form. Due to this reason, it is important to understand how a computer together with its peripheral devices handles data in its electronic circuits, on magnetic media and in optical devices.

Data representation in digital circuits

Electronic components, such as microprocessor, are made up of millions of electronic circuits. The availability of high voltage(on) in these circuits is interpreted as '1' while a low voltage (off) is interpreted as '0'. This concept can be compared to switching on and off an electric circuit. When the switch is closed the high voltage in the circuit causes the bulb to light ('1' state). on the

other hand when the switch is open, the bulb goes off ('0' state). This forms a basis for describing data representation in digital computers using the binary number system.

Data representation on magnetic media

The laser beam reflected from the land is interpreted, as 1. The laser entering the pit is not reflected. This is interpreted as 0. The reflected pattern of light from the rotating disk falls on a receiving photoelectric detector that transforms the patterns into digital form. The presence of a magnetic field in one direction on magnetic media is interpreted as 1; while the field in the opposite direction is interpreted as "0". Magnetic technology is mostly used on storage devices that are coated with special magnetic materials such as iron oxide. Data is written on the media by arranging the magnetic dipoles of some iron oxide particles to face in the same direction and some others in the opposite direction.

Data representation on optical media

In optical devices, the presence of light is interpreted as '1' while its absence is interpreted as '0'. Optical devices use this technology to read or store data. Take example of a CD-ROM, if the shiny surface is placed under a powerful microscope, the surface is observed to have very tiny holes called **pits**. The areas that do not have pits are called **land**.

Reason for Use of Binary System in Computers

It has proved difficult to develop devices that can understand natural language directly due to the complexity of natural languages. However, it is easier to construct electric circuits based on the binary or ON and OFF logic. All forms of data can be represented in binary system format. Other reasons for the use of binary are that digital devices are more reliable, small and use less energy as compared to analog devices.

Bits, bytes, nibble and word

- The terms bits, bytes, nibble and word are used widely in reference to computer memory and data size.
- **Bits:** can be defined as either a binary, which can be 0, or 1. It is the basic unit of data or information in digital computers.
- **Byte:** a group of bits (8 bits) used to represent a character. A byte is considered as the basic unit of measuring memory size in computer.
- **A nibble:** is half a byte, which is usually a grouping of 4 bytes.
- **Word:** two or more bits make a word. The term **word length** is used as the measure of the number of bits in each word. For example, a word can have a length of 16 bits, 32 bits, 64 bits etc.

Types of Data Representation

Computers not only process numbers, letters and special symbols but also complex types of data such as sound and pictures. However, these complex types of data take a lot of memory and processor time when coded in binary form. This limitation necessitates the need to develop better ways of handling long streams of binary digits. Higher number systems are used in computing to reduce these streams of binary digits into manageable form. This helps to improve the processing speed and optimize memory usage.

Number systems and their representation

- A number system is a set of symbols used to represent values derived from a common base or radix.
- As far as computers are concerned, number systems can be classified into two major categories:
 - decimal number system
 - binary number system
 - octal number system
 - hexadecimal number system

Decimal number system

The term decimal is derived from a Latin prefix deci, which means ten. Decimal number system has ten digits ranging from 0-9. Because this system has ten digits; it is also called a base ten number system or denary number system. A decimal number should always be written with a subscript 10 e.g. X_{10} . But since this is the most widely used number system in the world, the subscript is usually understood and ignored in written work. However, when many number systems are considered together, the subscript must always be put so as to differentiate the number systems.

The magnitude of a number can be considered using these parameters.

- Absolute value
- Place value or positional value
- Base value
 - ▲ The **absolute** value is the magnitude of a digit in a number. for example the digit 5 in 7458 has an absolute value of 5 according to its value in the number line.
 - ▲ The **place value** of a digit in a number refers to the position of the digit in that number i.e. whether; tens, hundreds, thousands etc.
 - ▲ The **total value** of a number is the sum of the place value of each digit making the number.
 - ▲ The **base value** of a number also known as the **radix**, depends on the type of the number systems that is being used. The value of any number depends on the radix. for example the number 100_{10} is not equivalent to 100_2 .

Binary Number System

It uses two digits namely, 1 and 0 to represent numbers. unlike in decimal numbers where the place value goes up in factors of ten, in binary system, the place values increase by the factor of 2. binary numbers are written as X_2 . consider a binary number such as 1011_2 . The right most digit has a place value of 1×2^0 while the left most has a place value of 1×2^3 .

Octal Number System

Consists of eight digits ranging from 0-7. the place value of octal numbers goes up in factors of eight from right to left.

Hexadecimal Number System

This is a base 16 number system that consists of sixteen digits ranging from 0-9 and letters A-F where A is equivalent to 10, B to 11 up to F which is equivalent to 15 in base ten system. The place value of hexadecimal numbers goes up in factors of sixteen.

- A hexadecimal number can be denoted using 16 as a subscript or capital letter H to the right of the number. For example, 94B can be written as 94B₁₆ or 94BH.

Further conversion of numbers from one number system to another

- To convert numbers from one system to another. The following conversions will be considered.
- Converting between binary and decimal numbers.
- Converting octal numbers to decimal and binary form.
- Converting hexadecimal numbers to decimal and binary form.

Converting binary numbers to decimal numbers

- To convert a binary number to a decimal number, we proceed as follows:
- First, write the place values starting from the right hand side.
- Write each digit under its place value.
- Multiply each digit by its corresponding place value.
- Add up the products. The answer will be the decimal number in base ten.

Example

Convert 101101_2 to base 10 (or decimal) number

Place value	2^5	2^4	2^3	2^2	2^1	2^0
Binary digits	1	0	1	1	0	1

Multiply each digit by its place value

$$N_{10} = (1 \cdot 2^5) + (0 \cdot 2^4) + (1 \cdot 2^3) + (1 \cdot 2^2) + (0 \cdot 2^1) + (1 \cdot 2^0)$$

$$N_{10} = 32 + 0 + 8 + 4 + 0 + 1$$

$$= 45_{10}$$

$$32 \cdot 1 = 32$$

$$16 \cdot 0 = 0$$

$$8 \cdot 1 = 8$$

$$4 \cdot 1 = 4$$

$$2 \cdot 0 = 0$$

$$1 \cdot 1 = 1$$

$$= 45_{10}$$

NB: remember to indicate the base subscript since it is the value that distinguishes the different systems.

- The binary equivalent of the fractional part is extracted from the products by reading the respective integral digits from the top downwards as shown by the arrow next page.
- Combine the two parts together to set the binary equivalent.

6 Computer Architecture

Convert 0.375_{10} into binary form

Read this digits

$$0.375 \times 2 = 0.750$$

$$0.750 \times 2 = 1.500$$

$$0.500 \times 2 = 1.000 \text{ (fraction becomes zero)}$$

$$\text{Therefore } 0.375_{10} = 0.011_2$$

NB: When converting a real number from binary to decimal, work out the integral part and the fractional parts separately then combine them.

Example

Convert 11.011_2 to a decimal number.

Solution

Convert the integral and the fractional parts separately then add them up.

$$2 \times 1 = 2.000$$

$$1 \times 1 = +1.000$$

$$3.000_{10}$$

$$\text{Weight} \quad 2^1 \ 2^0 \ .2^{-1} \ 2^{-2} \ 2^{-3}$$

$$\text{Binary digit} \quad 1 \ 1 \ .0 \ 1 \ 1$$

$$\text{Values in base 10} \ 2 \ 1 \ .0 \ 0.25 \ 0.125$$

$$0.50 \times 0 = 0.000$$

$$0.25 \times 1 = 0.250$$

$$0.125 \times 1 = +0.125$$

$$0.375_{10}$$

$$3.000_{10} + 0.375_{10} = 3.375_{10}$$

$$\text{Thus } 11.011_2 = 3.375_{10}$$

Converting a decimal fraction to binary

Divide the integral part continuously by 2. For the fractional part, proceed as follows:

Multiply the fractional part by 2 and note down the product

- Take the fractional part of the immediate product and multiply it by 2 again.
- Continue this process until the fractional part of the subsequent product is 0 or starts to repeat itself.

The following examples illustrate how to convert hexadecimal number to a decimal number

Example

Convert octal number 321_8 to its binary equivalent

Solution

Working from left to the right, each octal number is represented using three digits and the combined we get the final binary equivalent. Therefore:

$$3 = 011_2$$

$$2 = 010_2$$

$$1 = 001_2$$

Combining the three from left to right

3 2 1

011 010 001

 $321_8 = 011010001_2$ **Converting binary numbers to hexadecimal numbers**

- To convert binary numbers to their binary equivalents, simply group the digits of the binary number into groups of four from right to left e.g. 11010001. The next step is to write the hexadecimal equivalent of each group e.g.

1101- D

0001- 1

The equivalent of 11010001 is D1H or D1₁₆**Converting hexadecimal numbers to decimal number**

- To convert hexadecimal number to base 10 equivalent we proceed as follows:
 - First, write the place values starting from the right hand side.
 - If a digit is a letter such as 'A' write its decimal equivalent
- Multiply each hexadecimal digit with its corresponding place value and then add the products

The binary equivalent of the fractional part is extracted from the products by reading the respective integral digits from the top downwards as shown by the arrow next page. Combine the two parts together to set the binary equivalent.

Convert 0.375₁₀ into binary form

Read this digits

$0.375 \times 2 = 0.750$

$0.750 \times 2 = 1.500$

$0.500 \times 2 = 1.000$ (fraction becomes zero)

Therefore $0.375_{10} = 0.011_2$ **Converting octal numbers to decimal numbers**

- To convert a base 8 number to its decimal equivalent we use the same method as we did with binary numbers. However, it is important to note that the maximum absolute value of a octal digit is 7. For example 982 is not a valid octal number because digit 9 is not an octal digit, but 736₈ is valid because all the digits are in the range 0-7. Example shows how to convert an octal number to a decimal number.

ExampleConvert 512₈ to its base 10 equivalent**Solution**Place value 8² 8¹ 8⁰

64 8 1

Octal digit 5 1 2

Write each number under its place value as shown below

Multiply each number by its place value.

$$\begin{aligned}
 N_{10} &= (5 \times 8^2) + (1 \times 8^1) + (2 \times 8^0) \\
 &= (5 \times 64) + 8 + 2 \\
 &= 320 + 8 + 2
 \end{aligned}$$

$$N_{10} = 330_{10}$$

$$64 \times 5 = 320$$

$$8 \times 1 = 8$$

$$1 \times 2 = 2$$

$$330$$

$$\text{Therefore } 512_8 = 330_{10}$$

Converting octal numbers to binary numbers

To convert an octal number to binary, each digit is represented by three binary digits because the maximum octal digit i.e. 7 can be represented with a maximum of seven digits. See table:

Octal digit	Binary equivalents
0	000
1	001
2	010
3	011
4	100
5	101
6	110
7	111

Example

Convert the hexadecimal number 111_{16} to its binary equivalent.

Solution

Place each number under its place value.

$$16^2 \ 16^1 \ 16^0$$

$$1 \ 1 \ 1$$

$$256 \times 1 = 256$$

$$16 \times 1 = 16$$

$$1 \times 1 = 1$$

$$273$$

$$\text{Therefore } 111_{16} = 273_{10}$$

Example

Convert octal number 321_8 to its binary equivalent

Solution

Working from left to the right, each octal number is represented using three digits and then combined we get the final binary equivalent. Therefore:

$$3=011_2$$

$$2=010_2$$

$$1=001_2$$

Combining the three from left to right

$$3 \quad 2 \quad 1$$

$$011 \ 010 \ 001$$

$$321_8 = 011010001_2$$

Converting binary numbers to hexadecimal numbers

To convert binary numbers to their binary equivalents, simply group the digits of the binary number into groups of four from right to left e.g. 11010001. The next step is to write the hexadecimal equivalent of each group e.g.

$$1101 - D$$

$$0001 - 1$$

The equivalent of 11010001 is D1H or D1₁₆

Converting hexadecimal numbers to decimal number

To convert hexadecimal number to base 10 equivalent we proceed as follows:

First, write the place values starting from the right hand side.

- If a digit is a letter such as 'A' write its decimal equivalent
 - ▲ Multiply each hexadecimal digit with its corresponding place value and then add the products
 - ▲ The following examples illustrate how to convert hexadecimal number to a decimal number

Example

Convert the hexadecimal number 111₁₆ to its binary equivalent

Solution

Place each number under its place value.

$$16^2 \ 16^1 \ 16^0$$

$$1 \quad 1 \quad 1$$

$$256 \times 1 = 256$$

$$16 \times 1 = 16$$

$$1 \times 1 = +1$$

$$273$$

Therefore 111₁₆ = 273₁₀

Converting Hexadecimal Numbers to Binary Numbers

- Since F is equivalent to a binary number 1111₂ the hexadecimal number are therefore represented using 4 digits as shown in the table below:

Hexadecimal digit	Decimal equivalent	Binary equivalent
00	00	0000
01	01	0001
02	02	0010
03	03	0011
04	04	0100
05	05	0101
06	06	0110
07	07	0111
08	08	1000
09	09	1001
A	10	1010
B	11	1011
C	12	1100
D	13	1101
E	14	1110
F	15	1111

The simplest method of converting a hexadecimal number to binary is to express each hexadecimal digit as a four bit binary digit number and then arranging the group according to their corresponding positions as shown in example

Example

Convert 321_{16}

Hexadecimal digit 3 2 1

Binary equivalent 0011 0010 0001

Combining the three sets of bits, we get 001100100001₂

$321_{16} = 001100100001_2$

Example 2

Convert $5E6_{16}$ into binary

Hexadecimal digit

5 E 6

Binary equivalent 0101 1110 0110

$5E6_{16} = 010111100110_2$

1.2 BINARY ARITHMETIC OPERATIONS

In mathematics, the four basic arithmetic operations applied on numbers are addition, subtraction, multiplications and division. In computers, the same operations are performed inside the central processing unit by the arithmetic and logic unit (ALU). However, the arithmetic and logic unit cannot perform binary subtractions directly. It performs binary subtractions using a process known as multiplication and division, the arithmetic and logic unit uses a method called *shifting* before adding the bits.

Representation of Signed Binary Numbers

- In computer technology, there are three common ways of representing a signed binary number.
 - ▲ Prefixing an extra sign bit to a binary number.
 - ▲ Using ones complement.
 - ▲ Using twos complement.

Prefixing an Extra Sign Bit to a Binary Number

In decimal numbers, a signed number has a prefix "+" for a positive number e.g. $+27_{10}$ and "-" for a negative number e.g. -27 . However, in binary, a negative number may be represented by prefixing a digit 1 to the number while a positive number may be represented by prefixing a digit 0. For example, the 7-bit binary equivalent of 127 is 1111111_2 . To indicate that it is positive, we add an extra bit (0) to the left of the number i.e. $(0)1111111_2$.

To indicate that it is negative number we add an extra bit (1) i.e. $(1)1111111_2$.

The problem of using this method is that the zero can be represented in two ways i.e. $(0)0000000_2$ and $(1)0000000_2$.

1.3 COMPLIMENTS

The term complement refers to a *part* which together with another makes up a *whole*. For example in geometry two complementary angle (90°). The idea of complement is used to address the problem of signed numbers i.e. positive and negative.

One's Complement

In decimal numbers (0 to 9), we talk of nine's complement. For example the nine's complement of 9 is 0, that of 5 is 4 while 3 is 6. However, in binary numbers, the ones complement is the bitwise NOT applied to the number. Bitwise NOT is a unary operator (operation on only one operand) that performs logical negation on each bit. For example the bitwise NOT of 11002 is 00112 . 0's are negated to 1's while 1's are negated to 0's.

Two's complement

Twos complement, equivalent to tens complement in decimal numbers, is the most popular way of representing negative numbers in computer systems. The advantages of using this method are:

- There are no two ways of representing a zero as in the case with other two methods.
- Effective addition and subtraction can be done even with numbers that are represented with a sign bit without a need for circuitries to examine the sign of an operand.

The twos complement of a number is obtained by getting the ones complement then adding a 1. For example, to get the twos complement of a decimal number 45_{10} ,

First convert it to its binary equivalent then find its ones complement. Add a 1 to ones complement i.e.

$$45_{10} = 00101101_2$$

$$\begin{aligned}\text{Bitwise NOT (00101101)} &= 11010010 \\ \text{Two's complement} &= 11010010_2 + 1_2 \\ &= 11010011_2\end{aligned}$$

Binary addition

The five possible additions in binary are

$$0 + 0 = 0$$

$$0 + 1_2 = 1_2$$

$$1_2 + 0 = 1_2$$

$$1_2 + 1_2 = 10_2 \text{ (read as 0, carry 1)}$$

$$1_2 + 1_2 + 1_2 = 11_2 \text{ (read as 1, carry 1)}$$

Example 1

Find the sum of $111_2 + 011_2$

Solution

Arrange the bits vertically. 111

Working from the right to the left, we proceed as follows: $+ 011$

Step 1 $1_2 + 1_2 = 10_2$, (write down 0 and carry 1) 1010_2

Step 2 $1_2 + 1_2 + 1_2 = 11_2$, (add and carry over digit to $1 + 1$ in order to get $1 + 1$

$+ 1$. From the sum, write down digit one the carry Forward)

Step 3 $1_2 + 1_2 + 0_2 = 10_2$, (add the carry over digit to $1 + 0$ in order to get

$1 + 1 + 0$. since this is the last step, write down 10)

Therefore $111_2 + 011_2 = 1010_2$

This can be summarized in the table

1st number 1 1 1

2nd number 0 1 1

Carry digit - 1 1

sum 1 0 1 0

Example 2

Add the following binary number

10110_2

1011_2

$+ 111_2$

Solution

Add the first two numbers and then add the sum to the third number as follows:

Step 1 Step 2

10110_2 100001_2

$+ 1011_2$ $+ 111_2$

100001_2 101000_2

Binary Subtraction

Direct subtraction

The four possible subtractions in binary are:

1. $0 - 0 = 0$
 2. $1_2 - 0 = 1_2$
 3. $1_2 - 1_2 = 0$
 2. $10_2 - 1_2 = 1_2$ (borrow 1 from the next most significant digit to make 0 become 10_2)
- Hence, $10_2 - 1_2 = 1_2$

Subtraction using one's compliment

The main purpose of using ones compliment in computers is to perform binary subtraction. For example to get the difference in $5 - 3$, using the ones compliment, we proceed as follows:

1. Rewrite the problem as $5 + (-3)$ to show that the computer binary subtraction by adding the binary equivalent of 5 to ones compliment of 3.
2. Convert the absolute value of 3 into 8-bits equivalent i.e. 00000011_2 .
3. Take the ones compliment of 00000011_2 i.e. 11111100_2 which is the binary representation of -3_{10} .
4. Add the binary equivalent of 5 to ones compliment of 3 i.e.

$$\begin{array}{r} 00000101 \\ + 11111000 \\ \hline (1)00000001 \end{array}$$

Subtraction Using Two's Compliments

Like in ones compliment, the two's compliment of a number is obtained by negating a positive number to its negative counterpart. For example to get the difference in $5 - 3$, using two's compliment, we proceed as follows:

1. Rewrite the problem as $5 + (-3)$
2. Convert the absolute value of 3 into 8-bit binary equivalent i.e. 00000011 .
3. Take the ones compliment of 00000011 i.e. 11111100 .
4. add a 1 to the ones compliment i.e. 11111100 to get 11111101
5. add the binary equivalent of 5 to the two's compliment of 3 i.e.

$$\begin{array}{r} 00000101 \\ + 11111001 \\ \hline \end{array}$$

$(1)00000010$ Ignoring the overflow bit, the resulting number is 00000010 , which is directly read as a binary equivalent of $+2$.

Example

Using two's compliment

$31_{10} - 17_{10}$ in binary form.

Solution

$$\begin{array}{l} 17_{10} \text{ in binary } 00010001 \\ 1's \text{ compliment } 11101110 \\ 2's \text{ compliment } 11101111 \\ 31_{10} = 00011111_2 \\ 00011111 + 11101111 = (1)00001110_2 \end{array}$$

1.4 FIXED POINT REPRESENTATION

Positive integers, including zero, can be represented as unsigned numbers. However, to represent negative integers, we need a notation for negative values. In ordinary arithmetic, a negative number is indicated by a minus sign and a positive number by a plus sign. Because of hardware limitations, computers must represent everything with 1's and 0's, including the sign of a number. As a consequence, it is customary to represent the sign with a bit placed in the leftmost position of the number. The convention is to make the sign bit equal to 0 for positive and to 1 for negative.

In addition to the sign, a number may have a binary (or decimal) point. The position of the binary point is needed to represent fractions, integers, or mixed integer-fraction numbers. The representation of the binary point in the register is complicated by the fact that it is characterized by a position in the register. There are two ways of specifying the position of the binary point in a register: by giving it a fixed position or by employing a floating-point representation. The fixed-point method assumes that the binary point is always fixed on a certain position. The two positions most widely used are (1) a binary point in the extreme left of the register to make the stored number a fraction, and (2) a binary point in the extreme right of the register so that the binary point is not actually present, but its presence is assumed from the fact that the number stored in the register is treated as a fraction or as an integer. The floating-point representation uses a second register to store a number that designates the position of the decimal point in the first register. Floating-point representation is discussed further in the next section.

Binary Integer Representation

	12	-12
Signed-magnitude	0 0001100	1 0001100
Signed-1's complement	0 0001100	1 1110011
Signed-2's complement	0 0001100	1 1110100

Computer Arithmetic

- Signed-magnitude awkward
- Signed-1's complement has two representations for zero: -0 and $+0$
- Therefore, signed-2's complement is used

Arithmetic Addition

- Simply add the two numbers and discard any leftmost carry out bit
- Negative numbers are in 2's complement
 - ▲ Including the result!

Example

+7	0000 0111
+11	0000 1011
+18	0001 0010
+7	0000 0111
-11	1111 0101
-4	1111 1100

-7	1111 1001
+11	0000 1011
+4	0000 0100
-7	1111 1001
-11	1111 0101
-18	1110 1110

Arithmetic Subtraction

- minuend - subtrahend = result
- minuend + subtrahend's r's complement = result
- Simply add the two numbers and discard any leftmost carry out bit
- Negative numbers are in 2's complement
 - ▲ Including the result!

Example

- $(-5) - (-14) = 9$
- $(1111\ 1011) - (1111\ 0010) = 0000\ 1001$
- $(1111\ 1011) + (0000\ 1110) = 0000\ 1001$
- $\boxed{1}0000\ 1001 = 0000\ 1001$

Overflow Examples

+79	0 100 1111
+81	0 101 0001
+160	1 010 0000

↑
 Carry-in = 1
 Carry-out = 0

-79	1 011 0001
-81	1 010 1111
-160	0 110 0000

↑
 Carry-in = 1
 Carry-out = 0

Decimal Fixed-Point Representation

The representation of decimal numbers in register is a function of the binary code used to represent a decimal digit. A 4-bit decimal code requires four flip-flops for each decimal digit. The representation of 4385 in BCD requires 16 flip-flops, four flip-flops for each digit. The number will be represented in a register with 16 flip-flops as follows:

0100 0011 1000 0101

The procedures developed for the signed-2's complement system apply also to the signed-10's complement system for decimal numbers. Addition is done by adding all digits, including the sign digit, and discarding the end carry. Obviously, this assumes that all negative numbers are in 10's complement form. Consider the addition $(+375) + (-240) = +135$ done in the signed 10's complement system.

$$\begin{array}{r}
 0\ 375 \quad (0000\ 0011\ 0111\ 0101)_{\text{BCD}} \\
 +\ 9\ 760 \quad (1001\ 0111\ 0110\ 0000)_{\text{BCD}} \\
 \hline
 0\ 135 \quad (0000\ 0001\ 0011\ 0101)_{\text{BCD}}
 \end{array}$$

The subtraction of decimal numbers either unsigned or in the signed-10's complement system is the same as in the binary cases. Take the 10's complement of the subtrahend and add it to the minuend. Many computers have special hardware to perform arithmetic calculations directly with decimal numbers in BCD. The user of the computer can specify by programming instructions that the arithmetic operations be performed with decimal numbers directly without having to convert them to binary.

1.5 FLOATING POINT REPRESENTATION

Floating point is useful for representing a number in a wide range: very small to very large. It is widely used in the scientific world. Consider, the following FP representation of a number

Sign bit	Exponent E	significand F (also called mantissa)
+/-	x x x x	y y y y y y y y y y y y y y

In decimal it means (+/-) 1. yyyyyyyyyyyy $\times 10^{\text{xxxx}}$

In binary, it means (+/-) 1. yyyyyyyyyyyy $\times 2^{\text{xxxx}}$

(The 1 is implied)

IEEE 754 single-precision (32 bits)

s	xxxxxxxx	yyyyyyyyyyyyyyyyyyyyyyyyyyyyyy	Single precision
1	8	23 bits	

Largest = $1.111 \dots \times 2^{+127} \approx 2 \times 10^{+38}$

Smallest = $1.000 \dots \times 2^{-128} \approx 1 \times 10^{-38}$

These can be positive and negative, depending on s. (But there are exceptions too)

IEEE 754 double precision (64 bits)

S	exponent	significand
1	11 bits	52 bits

Largest = $1.111 \dots \times 2^{+1023}$

Smallest = $1.000 \dots \times 2^{-1024}$

Overflow and underflow in FP

An overflow occurs when the number is too large to fit in the frame. An underflow occurs when the number is too small to fit in the given frame.

How do we represent zero?

IEEE standards committee solved this by making zero a special case: if every bit is zero (the sign bit being irrelevant), then the number is considered zero.

Then how do we represent 1.0?

It should have been 1.0×2^0 (same as 1)! The way out of this is that the interpretation of the exponent bits is not straightforward. The exponent of a single-precision float is "shift-127" encoded (biased representation), meaning that the actual exponent is (xxxxxxx minus 127). So thankfully, we can get an exponent of zero by storing 127.

Exponent = 11111111 (i.e. 255) means $255 - 127 = 128$

Exponent = 01111111 (i.e. 127) means $127 - 127 = 0$

Exponent = 00000001 (i.e. 1) means $1 - 127 = -126$

The consequence of shift-127

Exponent = 00000000 (reserved for 0) can no more be used to represent the smallest number. We forego something at the low extreme of the spectrum of representable magnitudes, which should be 2^{-127}

It seems wise, to give up the smallest exponent instead of giving up the ability to represent 1 or zero!

More special cases

Zero is not the only "special case" float. There are also representations for positive and negative infinity, and for a not-a-number (NaN) value, for results that do not make sense (for example, non-real numbers, or the result of an operation like infinity times zero). How do these work? A number is infinite if every bit of the exponent is 1 (yes, we lose another one), and is NaN if every bit of the exponent is 1 plus any mantissa bits are 1. The sign bit still distinguishes +/-inf and +/-NaN. Here are a few sample floating point representations:

Exponent	Mantissa	Object
0	0	Zero
0	Nonzero	Denormalized number*
1-254	Anything	+/- FP number
255	0	+ / - infinity
255	Nonzero	NaN

* Any non-zero number that is smaller than the smallest normal number is a denormalized number. The production of a denormal is sometimes called gradual underflow because it allows a calculation to lose precision slowly when the result is small.

Floating Point Addition

Example using decimal

$$A = 9.999 \times 10^1, B = 1.610 \times 10^{-1}, A+B = ?$$

Step 1. Align the smaller exponent with the larger one.

$$B = 0.0161 \times 10^1 = 0.016 \times 10^1 \text{ (round off)}$$

Step 2. Add significands

$$9.999 + 0.016 = 10.015, \text{ so } A+B = 10.015 \times 10^1$$

18 Computer Architecture

Step 3. Normalize

$$A+B = 1.0015 \times 10^2$$

Step 4. Round off

$$A+B = 1.002 \times 10^2$$

Now, try to add 0.5 and -0.4375 in binary.

Floating Point Multiplication

Example using decimal

$$A = 1.110 \times 10^{10}, B = 9.200 \times 10^{-5} \quad A \times B = ?$$

Step 1. Exponent of $A \times B = 10 + (-5) = 5$

Step 2. Multiply significands $1.110 \times 9.200 = 10.212000$

Step 3. Normalize the product $10.212 \times 10^5 = 1.0212 \times 10^6$

Step 4. Round off

$$A \times B = 1.021 \times 10^6$$

Step 5. Decide the sign of $A \times B$ (+ x + = +) So, $A \times B = + 1.021 \times 10^6$

1.6 OTHER BINARY CODES

In computing, a single character such as a letter, a number or a symbol is represented by a group of bits. The number of bits per character depends on the *coding* scheme used. The most common coding schemes are: Binary Coded Decimal, Gray Code, Self Complementing Code, Weighted Code, Excess-3 Code, Extended Binary Coded Decimal Interchange Code (EBCDIC) and American Standard Code for Information Interchange (ASCII).

Binary Coded Decimal

Binary Coded Decimal is a 4-bit code used to represent numeric data only. For example, the number like 9 can be represented using Binary Coded Decimal as 1001_2 . It is mostly used in simple electronic devices like calculators and microwaves. This is because it makes it easier to process and display individual numbers on their Liquid Crystal Display (LCD) screens.

A standard Binary Coded Decimal, an enhanced format of Binary Coded Decimal, is a 6-bit representation scheme which can represent non-numeric characters. This allows 64 characters to be represented. For letter A can be represented as 110001_2 using standard Binary Coded Decimal.

Gray Code:

It was named after Frank Gray and was used as a solution guide for the tower of Hanoi problem. It is used in labeling the axes of K-maps and allows system designers to perform error checking.

Conversion from BCD to Gray

- MSB of the gray code and the binary code is the same.
- Going from left to right, add each adjacent pair of binary code bits to get the next gray code bit discarding the carry obtained.

Input

0	0	0	0	0	0	0	0
0	0	0	1	0	0	0	1
0	0	1	0	0	0	1	0
0	0	1	1	0	0	1	1
0	1	0	0	0	1	1	0
0	1	0	1	0	1	1	1
0	1	1	0	0	1	0	1
0	1	1	1	0	1	0	0
1	0	0	0	1	1	0	0
1	0	0	1	1	1	0	1

Table 1.1 Truth table for BCD to Gray code conversion

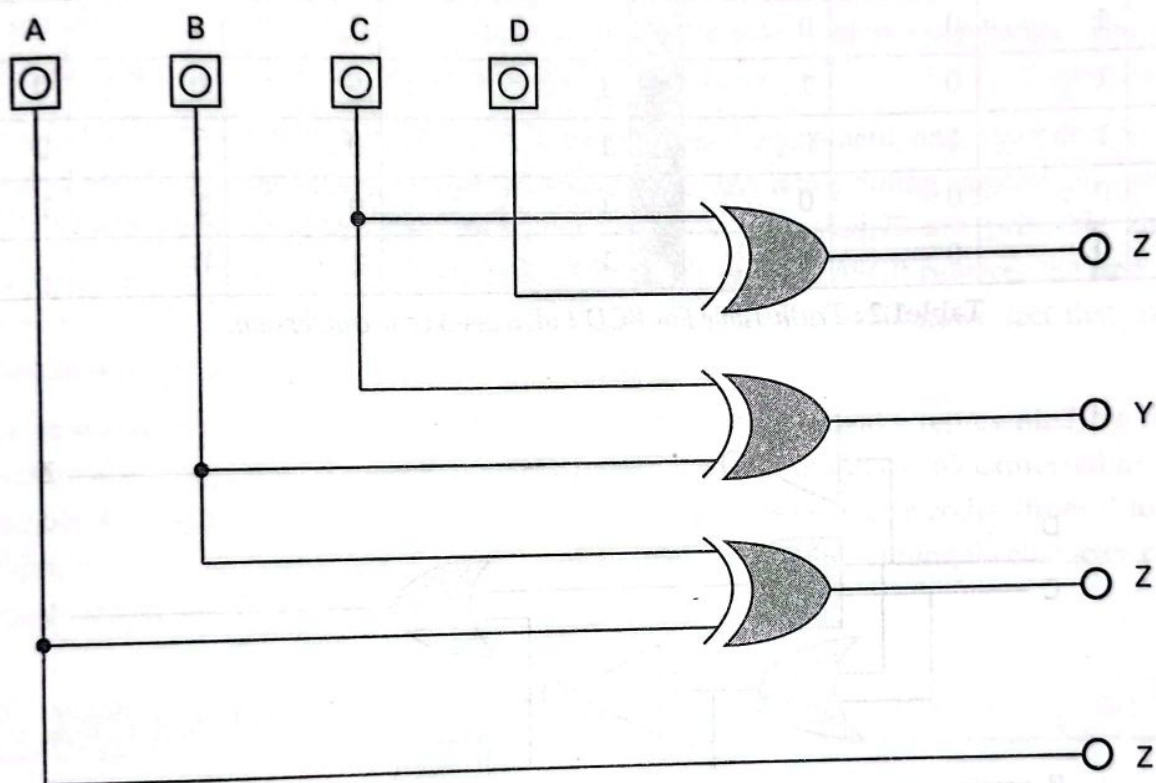


Figure 1.1 Logic Diagram for BCD to Gray code conversion

Self Complementing Code

The 2421, the excess - 3 and the 84-2-1 codes are examples of self - complementing codes. Such codes have the property that the 9's complement of a decimal number is obtained directly by changing 1's to 0's and 0's to 1's (i.e., by complementing each bit in the pattern). For example, decimal 395 is represented in the excess - 3 code as 0110 1100 1000. The 9's complement of 604 is represented as 1001 0011 0111, which is obtained simply by complementing each bit of the code (as with the 1's complement of binary numbers).

Excess-3 Code:

Excess- 3 code is also known as (XS-3), Stibitz code, biased representation or Excess-N and is complementary BCD code. It is used in older computers with a pre-specified number N and is represented by four bits as BCD value plus 3. The primary advantage of XS-3 coding over BCD

coding is that a decimal number can be nines' complement (for subtraction) as easily as binary number can be ones' complement

Input BCD				Output (Excess - 3)			
A	B	C	D	W	X	Y	Z
0	0	0	0	0	1	1	1
0	0	0	1	0	1	0	0
0	0	1	0	0	1	0	1
0	0	1	1	0	1	1	0
0	1	0	0	0	1	1	1
0	1	0	1	1	0	0	0
0	1	1	0	1	0	1	0
0	1	1	1	1	0	1	1
1	0	0	0	1	1	0	0
1	0	0	1	1	1	0	0

Table1.2 : Truth Table For BCD to Excess-3 code conversion.

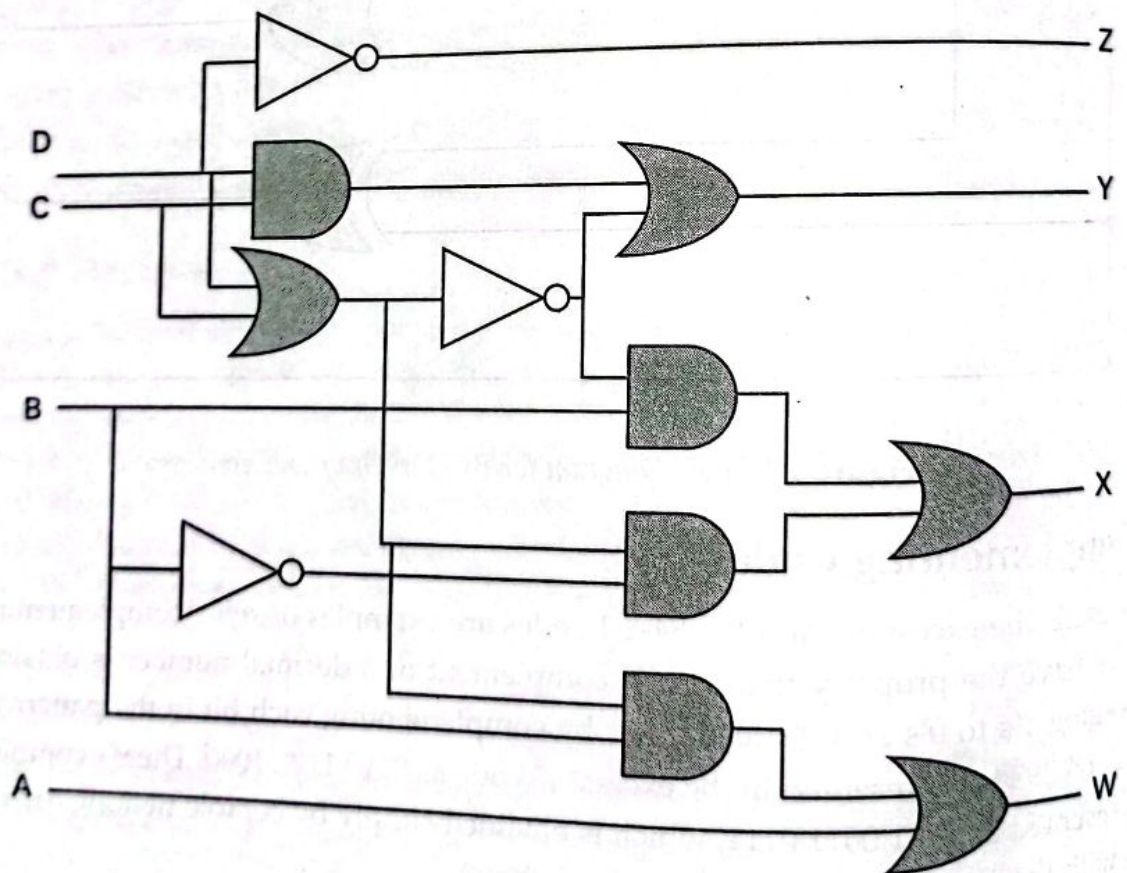


Figure 1.2 Logic diagram for BCD to Excess-3 code conversion.

Extended Binary Coded Decimal Interchange code (EBCDIC)

Extended Binary Coded Decimal Interchange code (EBCDIC) is an 8-bit character-coding scheme used primarily on IBM computers. A total of 256 (2^8) characters can be coded using this scheme. For example, the symbolic representation of letter A using Extended Binary Coded Decimal Interchange code is 11000001_2 .

American standard code for information interchange (ASCII)

American standard code for information interchange (ASCII) is a 7-bit code, which means that only 128 characters i.e. 2^7 can be represented. However, manufactures have added an eight bit to this coding scheme, which can now provide for 256 characters. This 8-bit coding scheme is referred to as an 8-bit American standard code for information interchange. The symbolic representation of letter A using this scheme is 1000001_2 .

This codes represent text in computers, communication equipment, and other devices that use text. It includes definitions for 128 characters: 33 are non-printing control characters(now mostly obsolete)that affect how text and space is processed;95 are printable characters, including the space, which is considered an invisible graphic ASCII reserves the first 32 codes (numbers 0-31 decimal) for control character and it is a very well-known fact that computers can manage internally only 0s (zeros) and 1s (ones).

For example, the ASCII code for the capital letter "A" is always represented by the order number 65, which is easily represent able using 0s and 1s in binary: 65 expressed as a binary number is 1000001 . The standard ASCII table defines 128 character codes (from 0 to 127), of which, the first 32 are control codes (non-printable), and the remaining 96 character codes are represent able character

1.7 ERROR DETECTION CODE

Binary information transmitted through some form of communication medium is subject to external noise that could change bits from 1 to 0, and vice-versa. An error detection code is a binary code that detects digital errors during transmission. The detected errors cannot be corrected but their presence is indicated. The usual procedure is to observe the frequency of errors. If errors occur infrequently at random, the particular erroneous information is transmitted again. If the error occurs too often, the system is checked for malfunction.

The most common error detection code used is the party bit. A party bit is an extra bit include with a binary message to make the total number of 1's either odd or even. A message of three bits and two possible parity bits is shown in table 3-7. The P(odd) bit is chosen in such a way as to make the sum of 1's even. In either case, the sum is taken over the message and the P bit. In any particular application, one or the other type of parity will be adopted. The even-parity scheme has the disadvantage of having a bit combination of all 0's, while in the odd parity there is always one bit (of the four bits that constitute the message and P) that is 1. Note that P(odd) is the complement of the P(even).

During transfer of information from one location to another, the parity bit is handled as follows. At the sending end, the message (in this case three bits) is applied to parity generator, where the required parity bit is generated. The message, including the parity bit, is transmitted to the destination. At the receiving end, all the oncoming bits (in this case, four) are applied to a parity checker that checks the proper parity adopted (odd or even). An error is detected if the checker parity does not conform to the adopted parity. The parity method detects the presence of one, three, or any odd number of errors. An even number of errors is not detected.

Message xyz	P(odd)	P(even)
000	1	0
001	0	1
010	0	1
011	1	0
100	0	1
101	1	0
110	1	0
111	0	1

Parity generator and checker networks are logic circuits constructed with exclusive-OR functions. This is because the exclusive-OR function of three or more variables is by definition an odd function. An odd function is a logic function whose value is binary 1 if, and only if, an odd number of variables are equal to 1. According to this definition, the P(even) function is the exclusive-OR of x , y , and z because it is equal to 1 when either one or all three of the variables are equal to 1. The P(odd) function is the complement of the P(even) function.

As an example, consider a 3-bit message to be transmitted with an odd parity bit. At the sending end, the odd-parity bit is generated by a parity generator circuit. As shown in Fig. 1, this circuit consists of one exclusive-OR and one exclusive-NOR gate. Since P(even) is the exclusive-OR of x , y , z and P(odd) is the complement of P(even), it is necessary to employ an exclusive-NOR gate for the needed complementation. The message and the odd-parity bit are transmitted to their destination where they are applied to a parity checker. An error has occurred during transmission of the parity of the four bits received is even, since the binary information transmitted was originally odd. The output of the parity checker would be 1 when an error occurs, that is, when the number of 1's in the four inputs is even. Since the exclusive-OR function of the four inputs is an odd function, we again need to complement the output using an exclusive-NOR gate.

It is worth noting that the parity generator can use the same circuit as the parity checker if the fourth input is permanently held at a logic-0 value. The advantage of this is that the same circuit can be used for both parity generation and parity checking.

It is evident from the example above that even-parity generators and checkers can be implemented with exclusive-OR functions. Odd-parity networks need an exclusive-NOR at the output to complement the function.

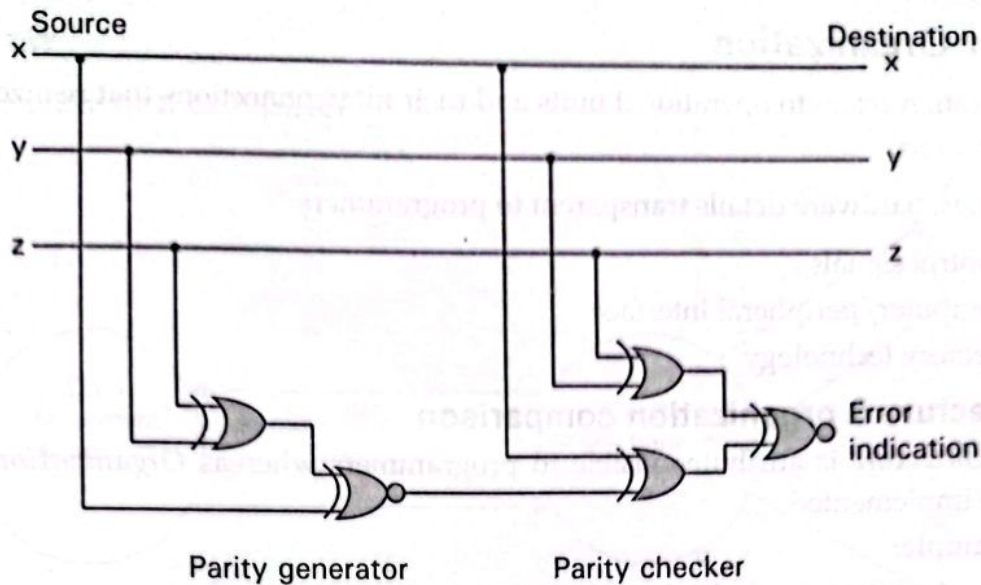


Fig. 1-3: Error detection with odd parity bit

1.8 COMPUTER ARCHITECTURE AND ORGANIZATION

Computer Architecture

Some Definition of Computer Architecture

"The design of integrated system which provides a useful tool to the programmer." (Baer)

"The study of the structure, behaviour, and design of computers." (Hayes)

"The design of the system specification at a general or subsystem level." (Abd-Alla)

"The art of designing a machine that will be a pleasure to work with." (Foster)

"The interface between the hardware and the lowest level software." (Hennessy and Patterson)

▪ Keyword:

- ▲ Design
- ▲ System
- ▲ Structure
- ▲ Art Tool Interface

Therefore, computer architecture refers to

- Attributes of a system *visible* to programmers
- Attributes that have a direct impact on the execution of programs

Attributes

- Instruction set
- Data representation
- I/O mechanisms
- Addressing techniques

Computer Organization

Organization refers to operational units and their interconnections that realize the architecture specifications.

Attributes: hardware details transparent to programmers

- Control signals
- Computer/peripheral interface
- Memory technology

Architecture & organization comparison

- *Architecture* is attributes visible to programmers whereas *Organization* is how features are implemented
- Example:
 - ▲ Architecture: multiply instruction?
 - ▲ Organization: hardware multiply unit or done by repeated addition? (how implemented?)
- Example:
 - ▲ IBM/370 architecture
 - ▲ different models (organizations)
- Family Concept
 - ▲ All Intel x86 family share the same basic architecture
 - ▲ The IBM System/370 family share the same basic architecture
 - ▲ This gives code compatibility (at least backwards)
- Organization differs between different versions

Structure & Function

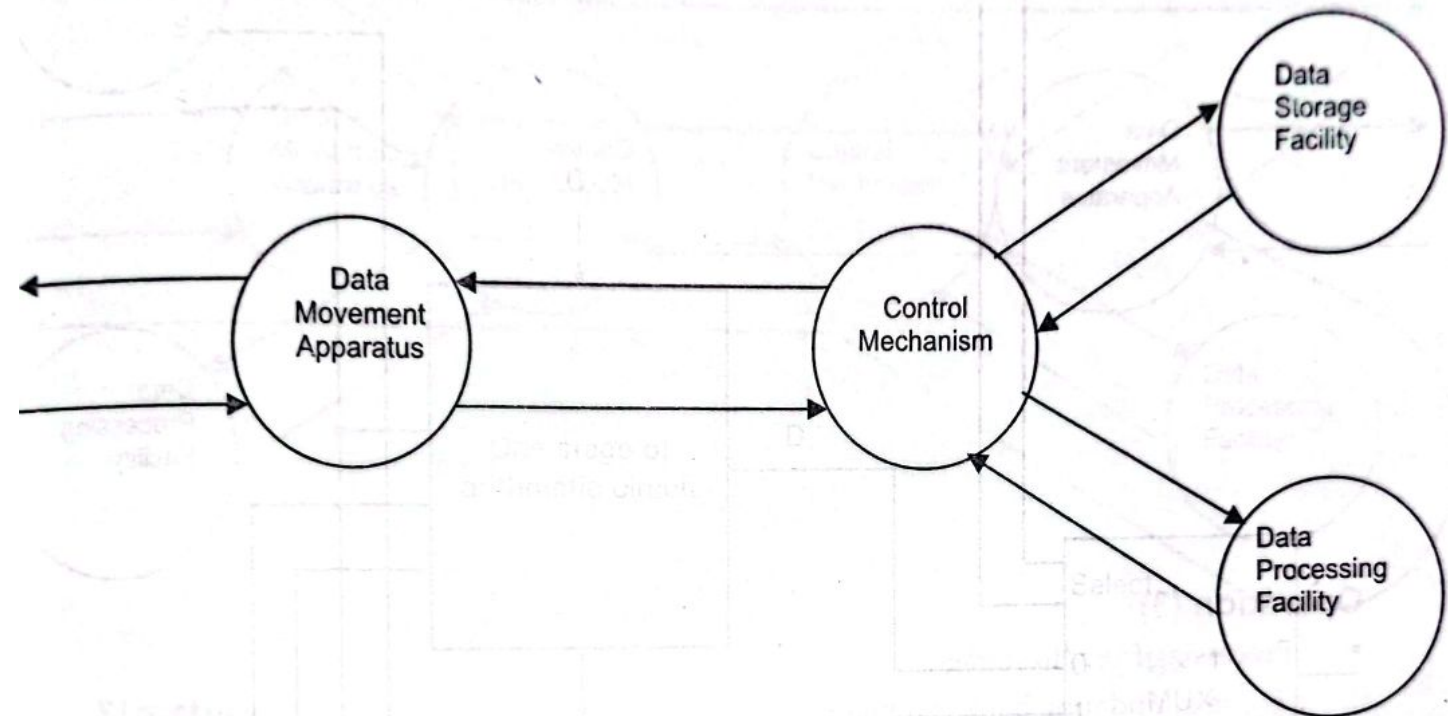
- Computer is a complex system that's why its description and design is always challenging
- Hierarchic system:
 - ▲ A set of interrelated subsystems, each subsystem hierarchic in structure until lowest level of elementary subsystems is reached
- At each level of the system, the designer is concerned with structure and function.
- Structure is the way in which components relate to each other
- Function is the operation of individual components as part of the structure

Function

- Basic computer functions:
 - ▲ Data processing
 - ▲ Data storage
 - ▲ Data movement
 - ▲ Control

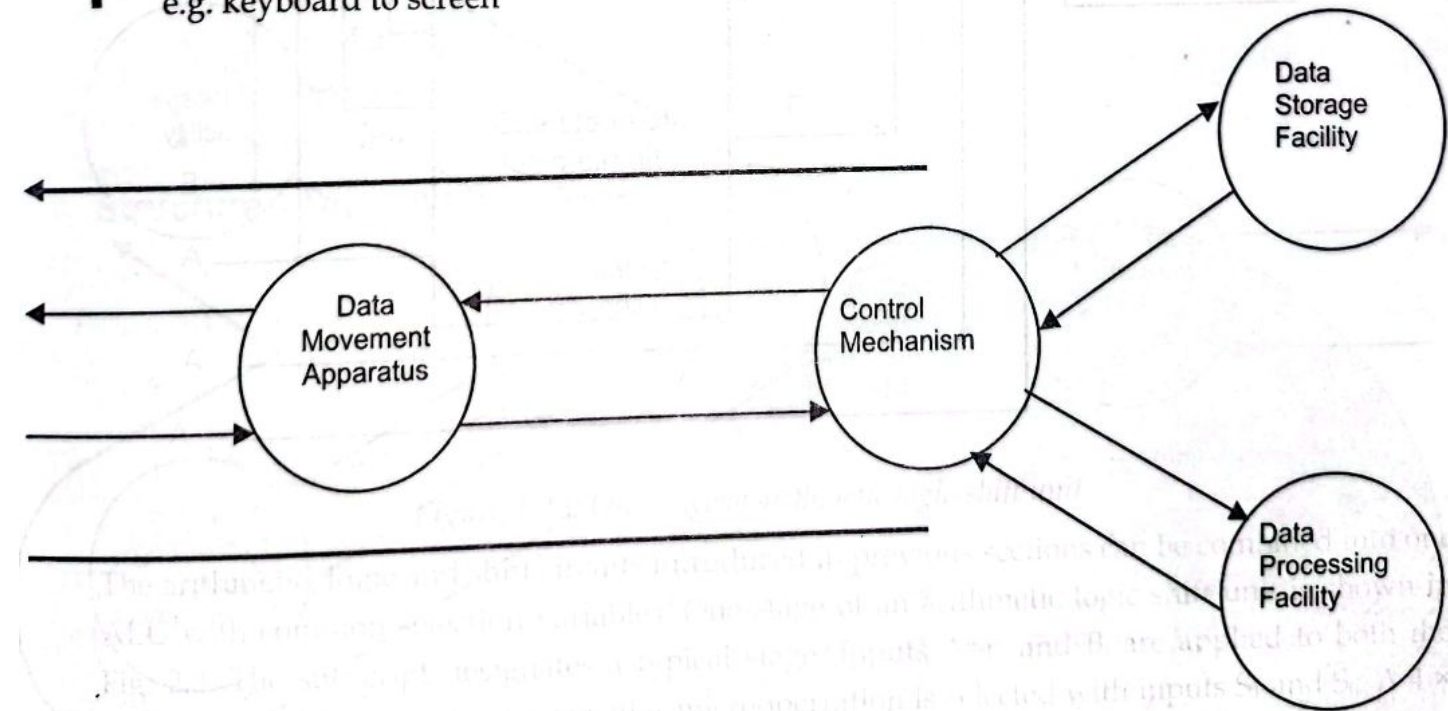
Functional view

Functional view of a computer



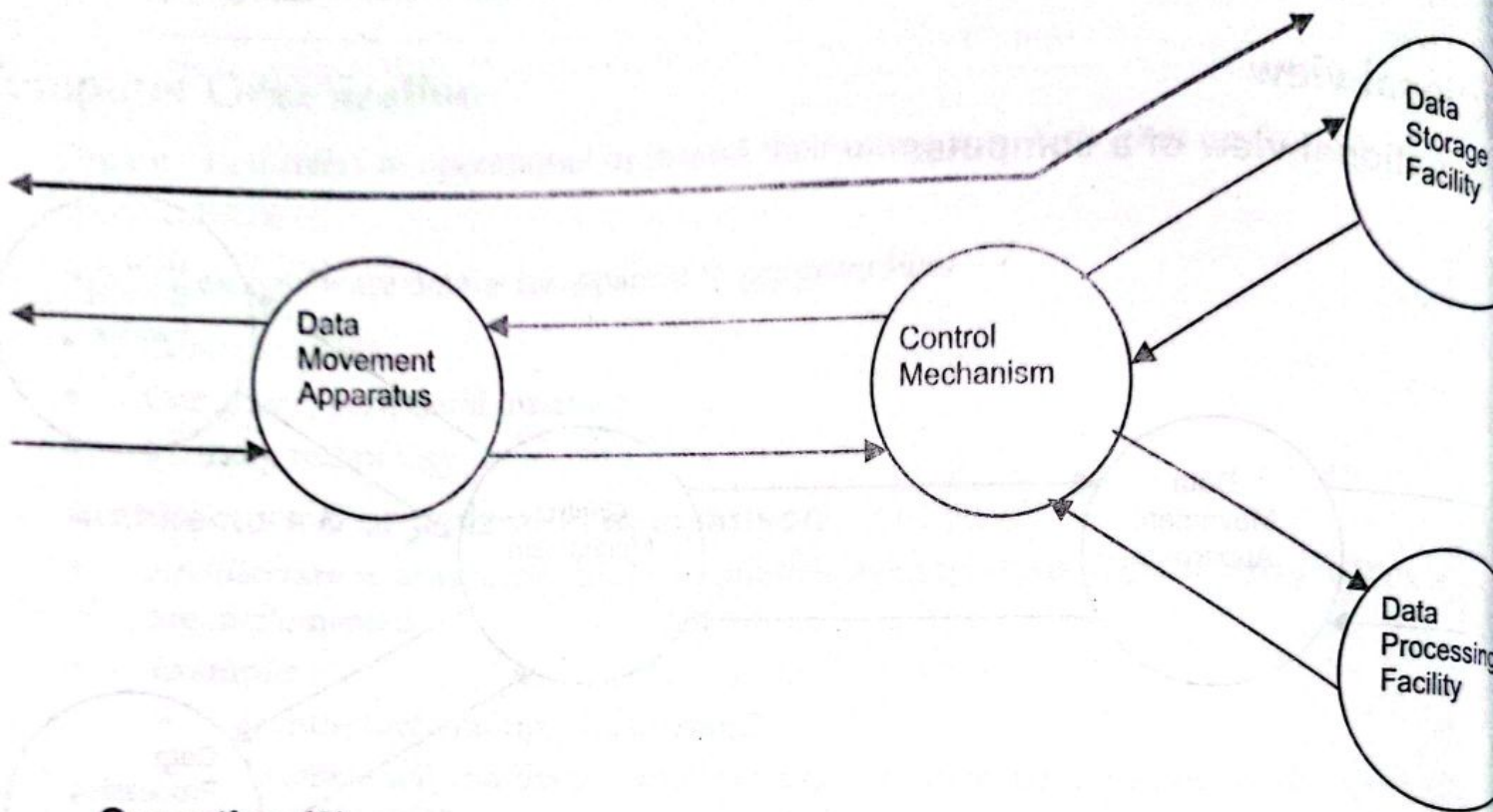
Data movement

- e.g. keyboard to screen



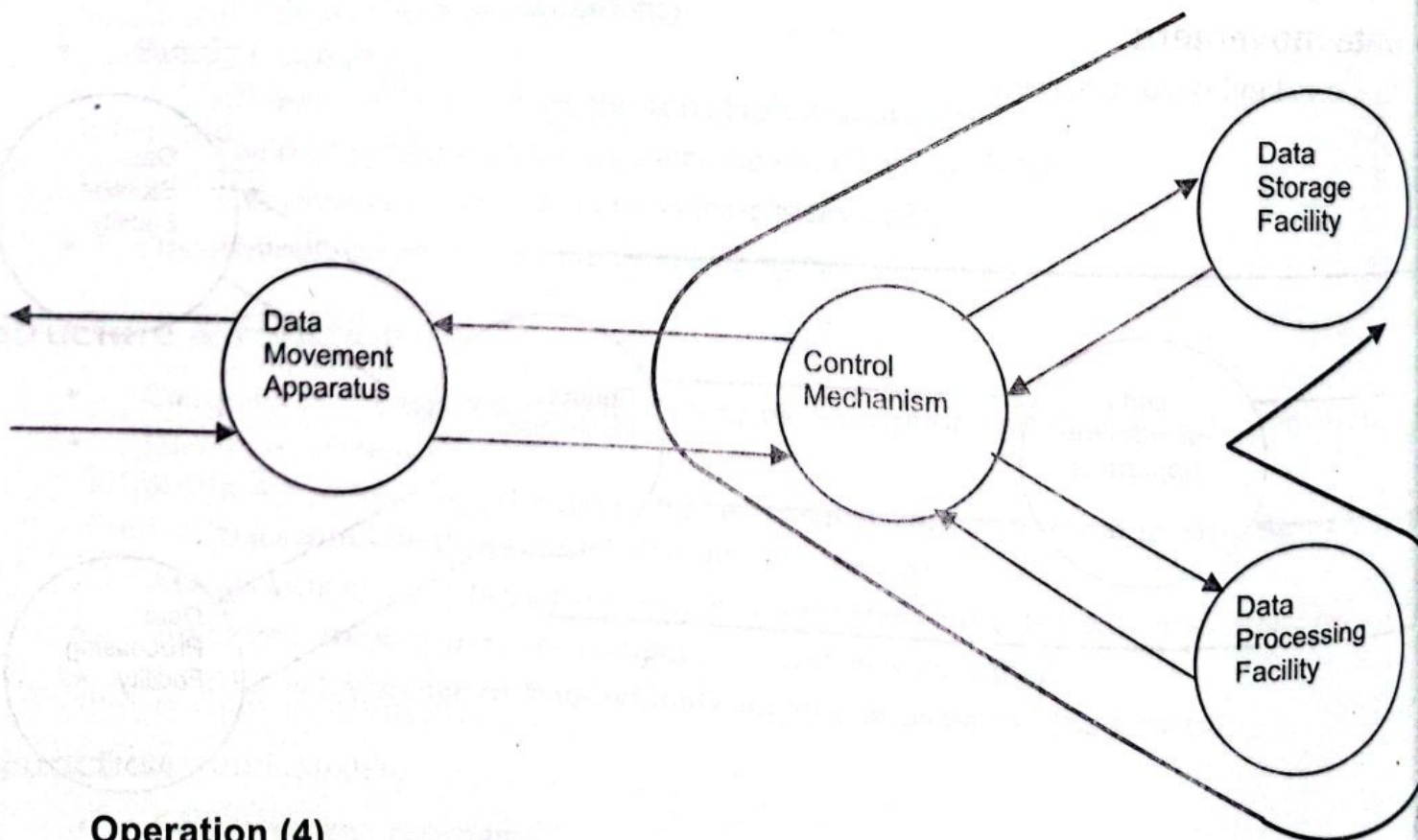
Storage

- e.g. Internet download to disk



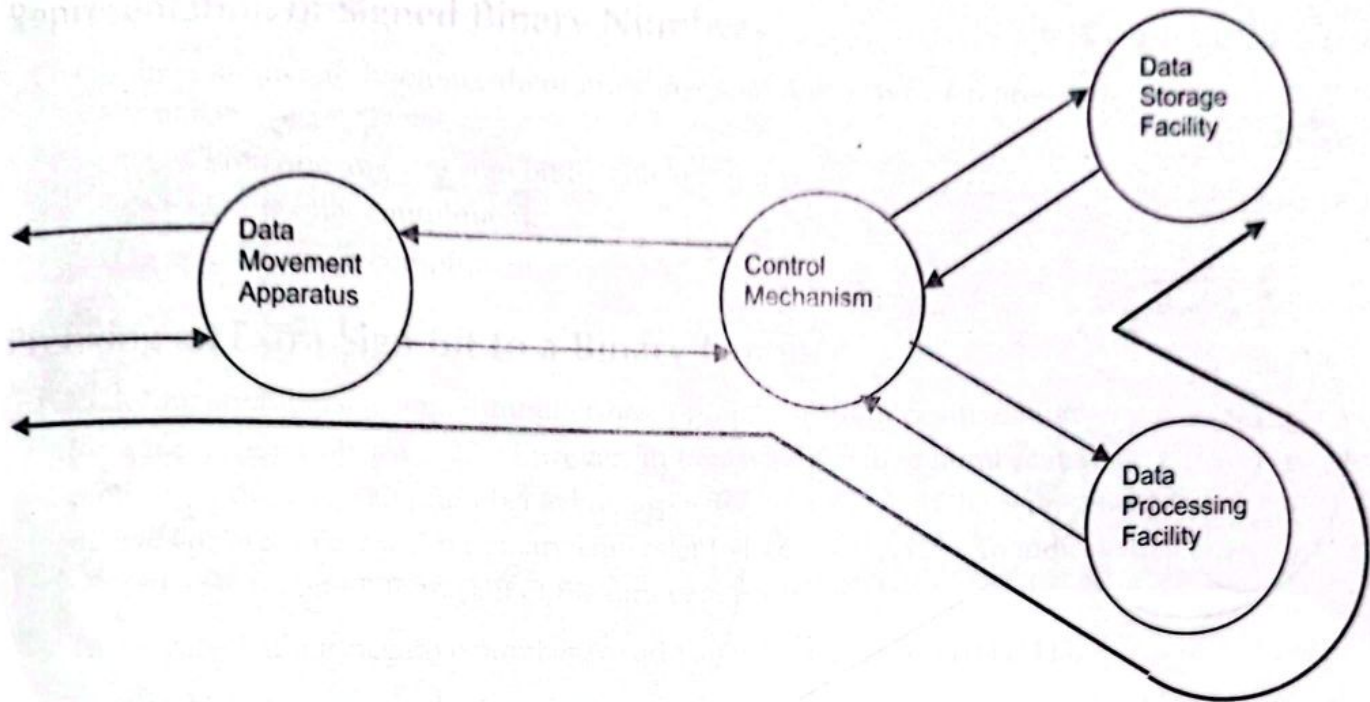
Operation (3)

- Processing from/to storage
 - e.g. updating bank statement



Operation (4)

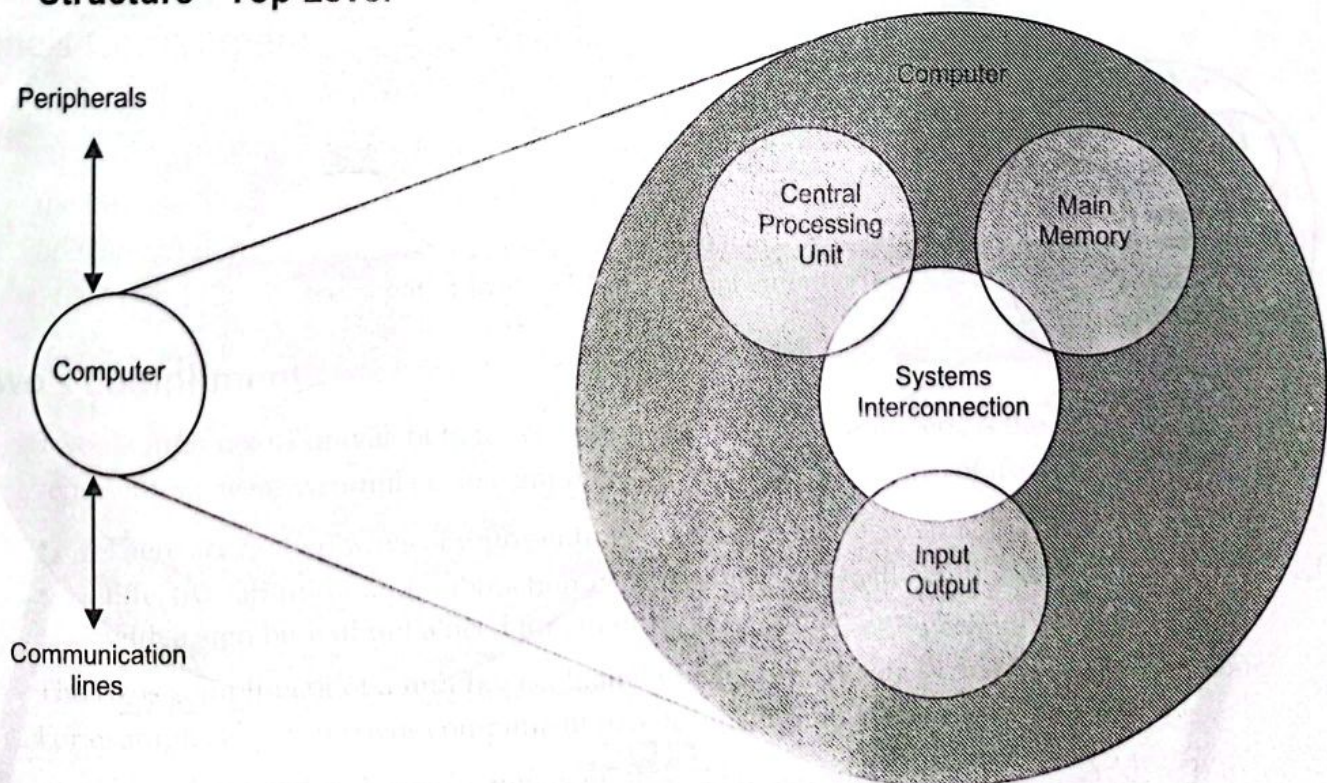
- Processing from storage to I/O
 - e.g. printing a bank statement



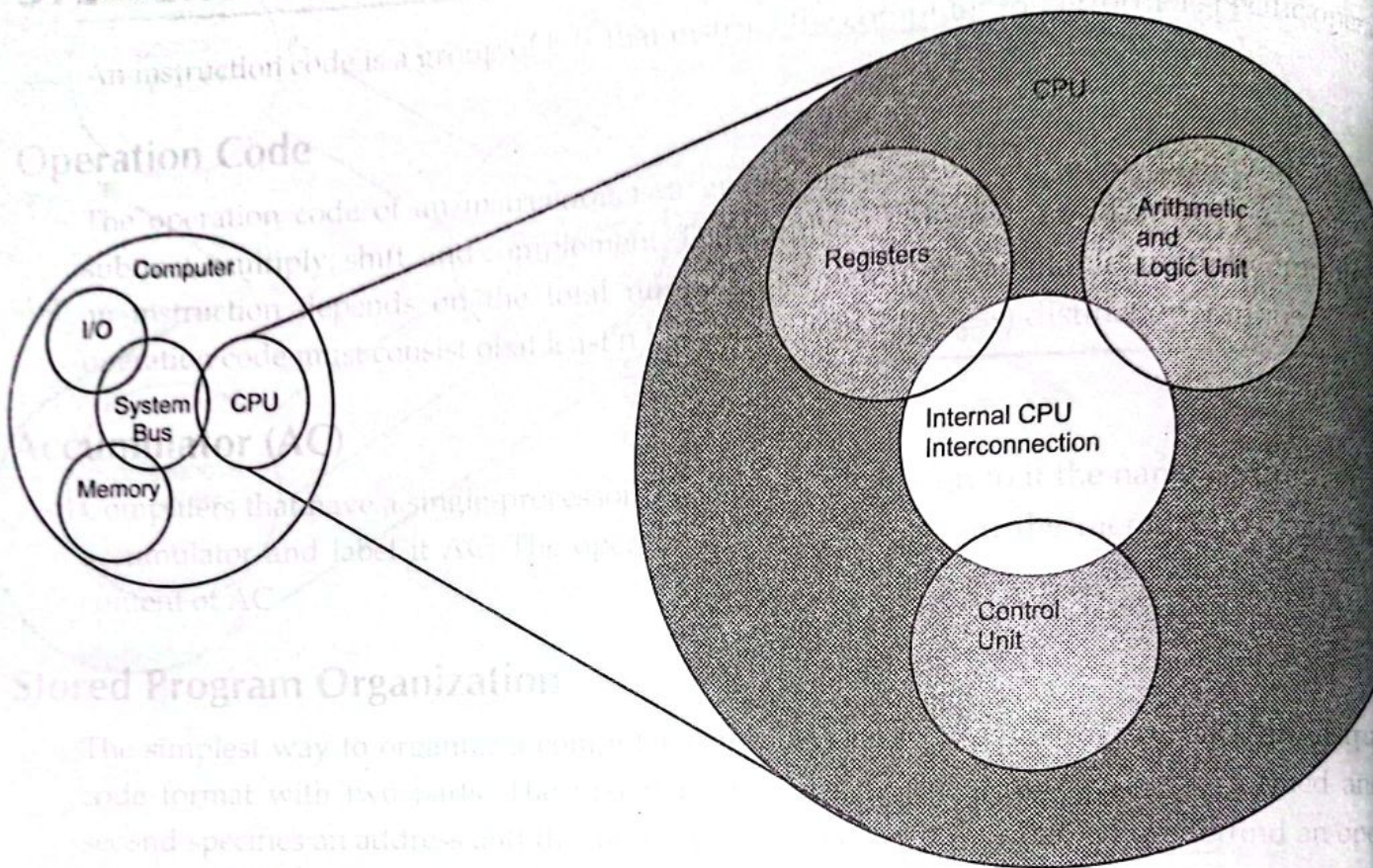
Structure

- Four main structural components
 - ▲ CPU
 - ▲ Main Memory
 - ▲ I/O Devices
 - ▲ System Interconnection

Structure - Top Level



3. Structure - The CPU



Structure - The Control Unit

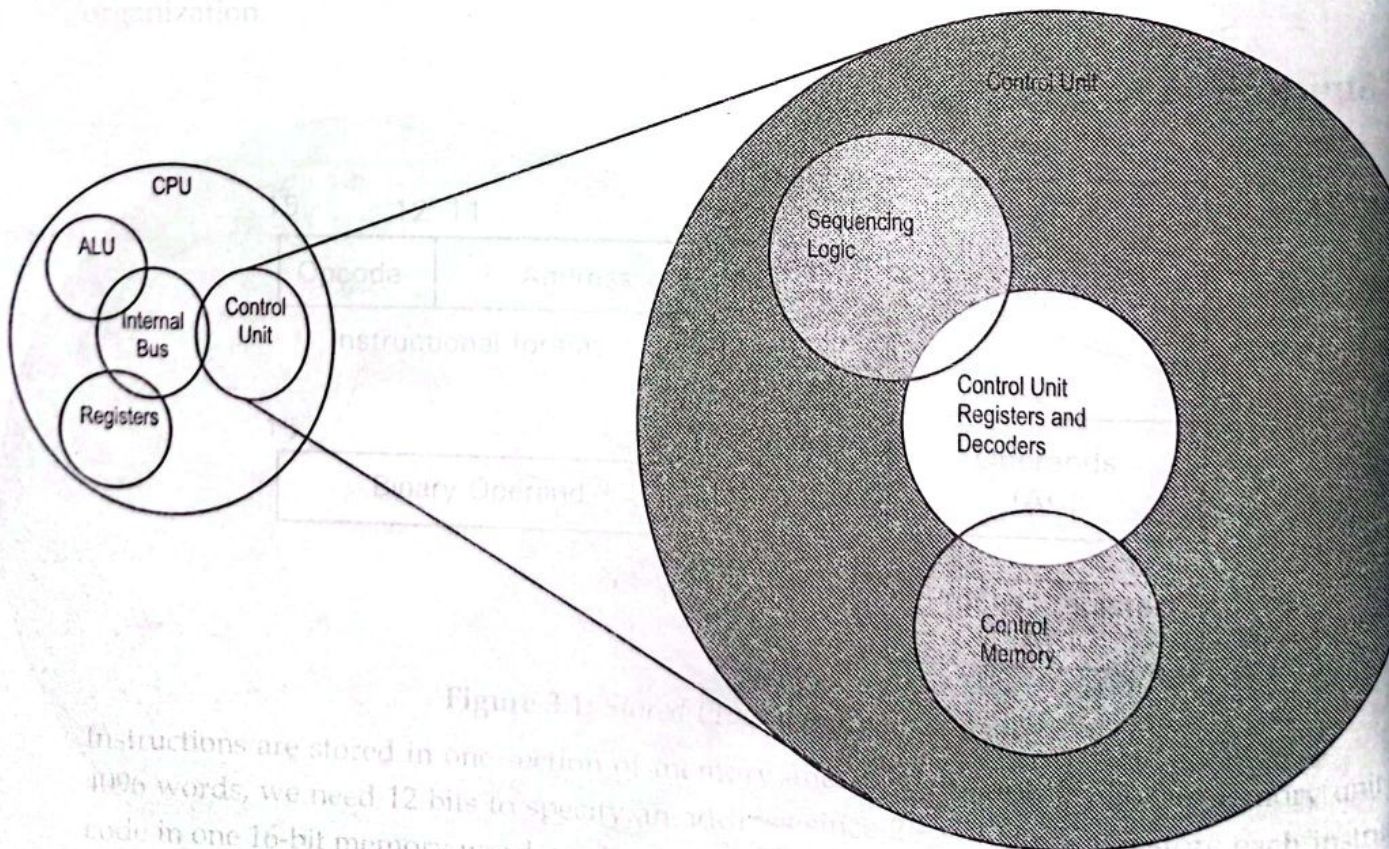


Figure 3.1: Stored Program Organization

Instructions are stored in one section of memory. If we have 1000 words, we need 12 bits to specify an address. Since each instruction is 16 bits long, we have 16 bits available for the instruction code. To specify one out of 16 possible instructions, we need 4 bits for the operation code (abbreviated as 'opcode'). The control reads a 16-bit instruction from the program portion of memory.